

Wydział Informatyki Politechniki Białostockiej

Systemy wbudowane

Tytuł: Zapoznanie się z CAD Quartus i płytą prototypową DE1-SoC

Wykonujący: Łukasz Modzelewski, Mariusz Rostkowski

PS 1

Zespół 4

Praca laboratoryjna 1

Studia dzienne

Informatyka

Semestr VI

Prowadzący: prof. dr hab. inż. Valery Salaouyou

Data: 03.03.2022

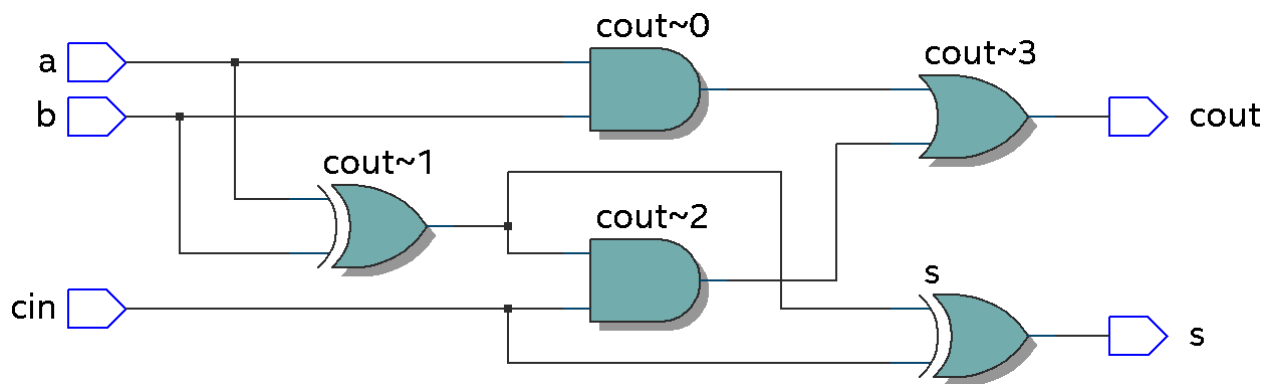
Zadanie 1

Zaimplementuj projekt jednobitowego sumatora przy pomocy operatorów **assign**.

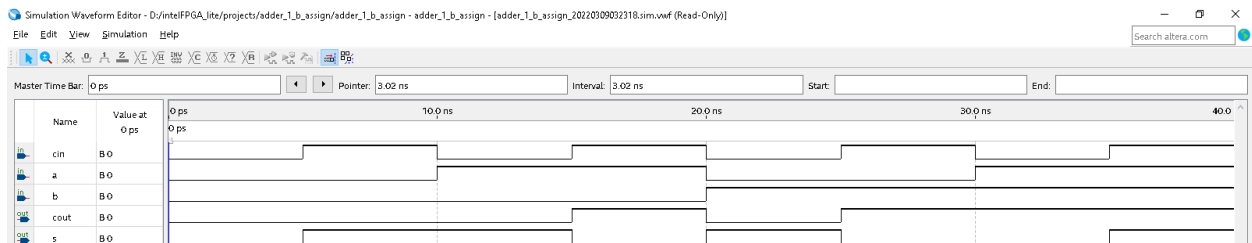
Kod w Verilog

```
module adder_1_b_assign(  
    input cin, a, b,  
    output cout, s);  
  
    assign s = cin ^ (a ^ b);  
    assign cout = a & b | (a ^ b) & cin;  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



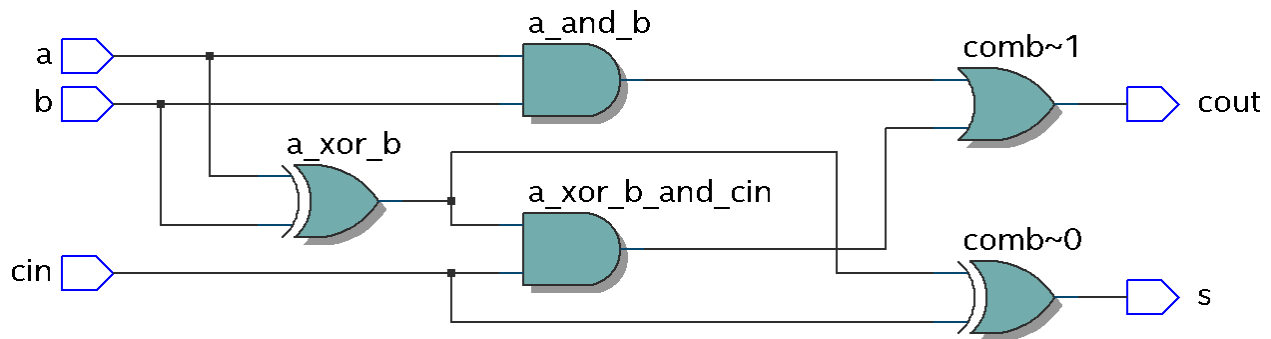
Zadanie 2

Zaimplementuj projekt jednobitowego sumatora za pomocą prymitywów języka Verilog.

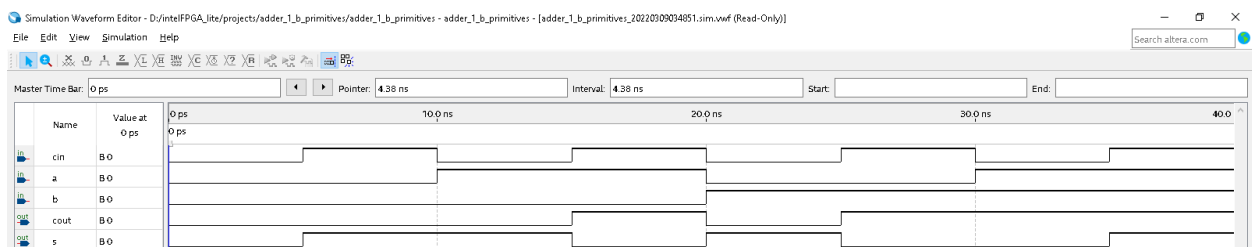
Kod w Verilog

```
module adder_1_b_primitives(  
    input cin, a, b,  
    output cout, s);  
  
    wire a_xor_b;  
    xor(a_xor_b, a, b);  
    xor(s, cin, a_xor_b);  
  
    wire a_and_b, a_xor_b_and_cin;  
    and(a_and_b, a, b);  
    and(a_xor_b_and_cin, a_xor_b, cin);  
    or(cout, a_and_b, a_xor_b_and_cin);  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



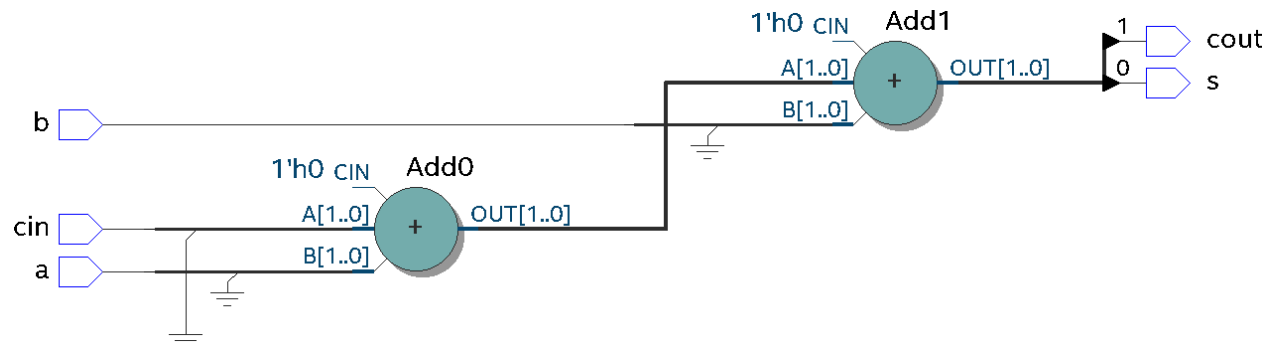
Zadanie 3

Zaimplementuj projekt jednobitowego sumatora za pomocą operacji arytmetycznej „+”.

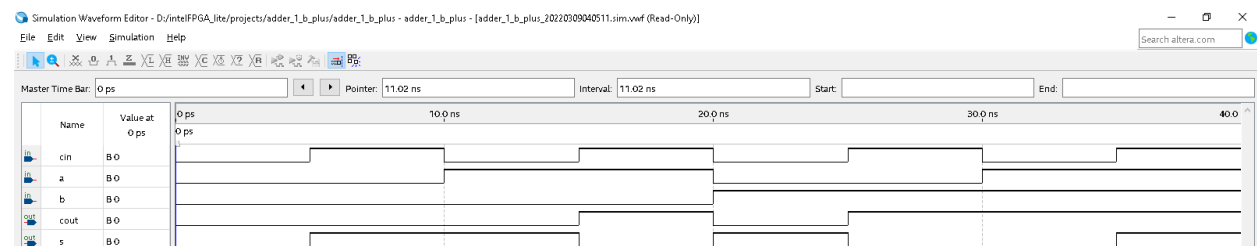
Kod w Verilog

```
module adder_1_b_plus(  
    input cin, a, b,  
    output cout, s);  
  
    assign {cout, s} = cin + a + b;  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Zadanie 4

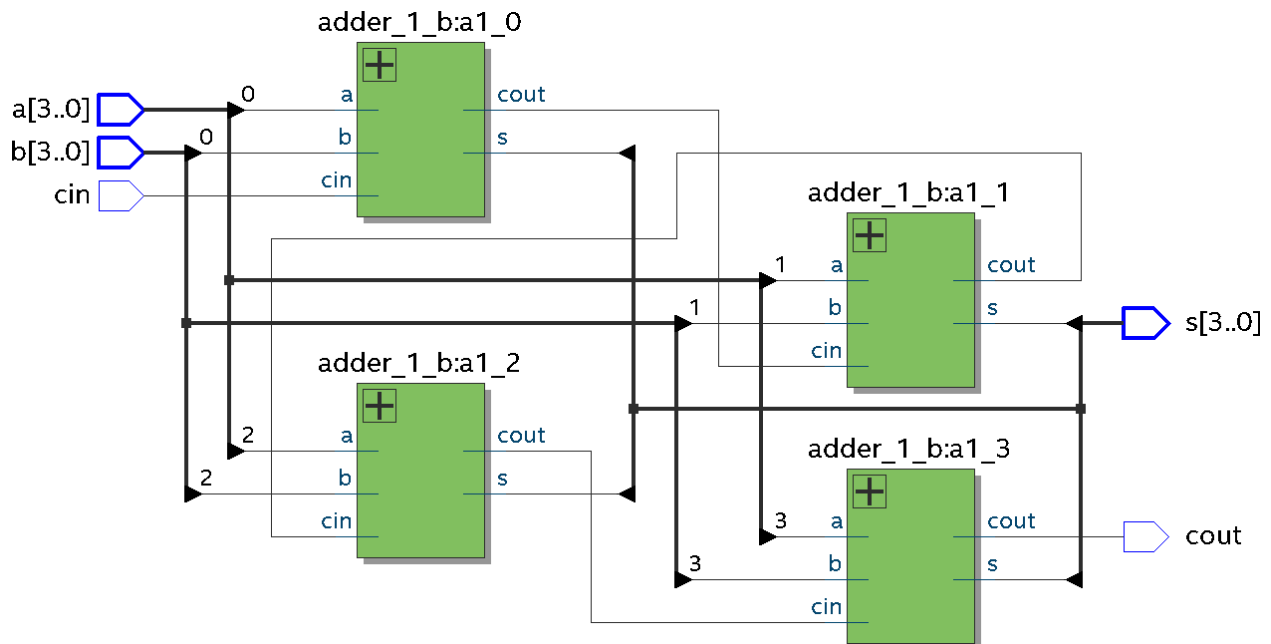
Zaimplementuj 4-bitowy iteracyjny sumator z przeniesieniami (ripple carry) z następującymi pinami:

Wyprowadzenia	Piny
A[3:0]	SW[3:0]
B[3:0]	SW[7:4]
<u>cin</u>	SW[9]
S[3:0]	LEDR[3:0]
<u>cout</u>	LEDR[9]

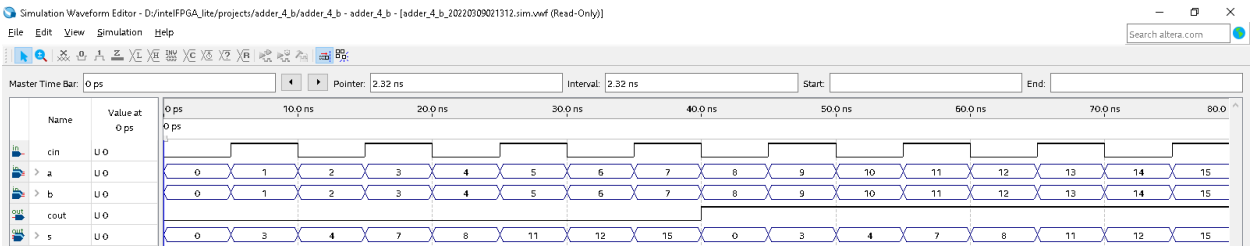
Kod w Verilog

```
module adder_4_b(  
    input cin,  
    input [3:0] a, b,  
    output cout,  
    output [3:0] s);  
  
    wire [2:0] c;  
    adder_1_b a1_0(cin, a[0], b[0], c[0], s[0]);  
    adder_1_b a1_1(c[0], a[1], b[1], c[1], s[1]);  
    adder_1_b a1_2(c[1], a[2], b[2], c[2], s[2]);  
    adder_1_b a1_3(c[2], a[3], b[3], cout, s[3]);  
  
endmodule  
  
module adder_1_b(  
    input cin, a, b,  
    output cout, s);  
  
    assign s = cin ^ (a ^ b);  
    assign cout = a & b | (a ^ b) & cin;  
  
endmodule
```

Wyniki syntezy



Wyniki symulacji funkcjonalnej



Wnioski

Wszystkie zadania zostały zrealizowane.